



PDF Download
3638530.3664112.pdf
06 April 2026
Total Citations: 2
Total Downloads: 416

Latest updates: <https://dl.acm.org/doi/10.1145/3638530.3664112>

RESEARCH-ARTICLE

Eventually, all you need is a simple evolutionary algorithm (for neuroevolution of continuous control policies)

MICHEL EL SALIBY, University of Trieste, Trieste, TS, Italy

GIORGIA NADIZAR, University of Trieste, Trieste, TS, Italy

ERICA SALVATO, University of Trieste, Trieste, TS, Italy

ERIC MEDVET, University of Trieste, Trieste, TS, Italy

Open Access Support provided by:

University of Trieste

Published: 01 August 2024

Citation in BibTeX format

GECCO '24 Companion: Genetic and Evolutionary Computation Conference Companion

July 14 - 18, 2024
VIC, Melbourne, Australia

Conference Sponsors:
SIGEVO

Eventually, All You Need is a Simple Evolutionary Algorithm (for Neuroevolution of Continuous Control Policies)

Michel El Saliby
DIA, University of Trieste
Trieste, Italy

michel.elsaliby@studenti.units.it

Erica Salvato
DIA, University of Trieste
Trieste, Italy
erica.salvato@dia.units.it

Giorgia Nadizar
MIGe, University of Trieste
Trieste, Italy
giorgia.nadizar@phd.units.it

Eric Medvet
DIA, University of Trieste
Trieste, Italy
emedvet@units.it

ABSTRACT

Artificial neural networks (ANNs) are a popular choice for tackling continuous control tasks due to their approximation abilities. When the ANN architecture is fixed, finding optimal weights becomes a numerical optimization problem, suitable for evolutionary algorithms (EAs), *i.e.*, a form of neuroevolution. Here, we compare the performance of well-established EAs in solving neuroevolution problems, focusing on continuous control. We evaluate them on a set of navigation problems and a set of control problems based on modular soft robots. As a reference, we compare the same EAs on regression problems and classic numerical optimization benchmarks. Our findings suggest that simple EAs like genetic algorithm (GA) and differential evolution (DE) achieve good performance on control problems, even if they are surpassed by more sophisticated algorithms on benchmark problems. We hypothesize that the effectiveness of these simpler EAs stems from their use of crossover, which can be advantageous in the rugged fitness landscapes encountered in complex control tasks.

CCS CONCEPTS

• **Theory of computation** → *Evolutionary algorithms*; • **Computing methodologies** → *Evolutionary robotics*; *Neural networks*.

KEYWORDS

Neuroevolution, Continuous control, Policy search

ACM Reference Format:

Michel El Saliby, Giorgia Nadizar, Erica Salvato, and Eric Medvet. 2024. Eventually, All You Need is a Simple Evolutionary Algorithm (for Neuroevolution of Continuous Control Policies). In *Genetic and Evolutionary Computation Conference (GECCO '24 Companion)*, July 14–18, 2024, Melbourne, VIC, Australia. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3638530.3664112>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

GECCO '24 Companion, July 14–18, 2024, Melbourne, VIC, Australia

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0495-6/24/07...\$15.00

<https://doi.org/10.1145/3638530.3664112>

1 INTRODUCTION AND RELATED WORKS

Evolutionary algorithms (EAs) have emerged as valuable tools for optimization across diverse domains. In neuroevolution, where EAs are applied to evolve artificial neural networks (ANNs), continuous control tasks stand out as particularly challenging due to their complex and high-dimensional search spaces. However, by harnessing the principle of natural selection and genetic variation, EAs proved to be a promising alternative to data-driven control [36, 37] and reinforcement learning (RL) [4] approaches.

Previous studies comparing ANNs controllers evolved through EAs primarily focused on specific domains, such as Atari games [11], devices fault diagnosis [43], or robot navigation [17]. Moreover, they often relied on pairwise comparisons, hindering the assessment of general performance trends [39]. Notably, several studies compared neuroevolution to RL approaches in continuous control [21, 35, 40], a dichotomy which is decades old [13, 29, 46]. What is still unclear is whether a sophisticated EA is needed to solve a neuroevolution problem on a complex control task or if complex control problems can be tackled effectively with simple EAs.

In this paper, we attempt to fill this gap by presenting a thorough comparative analysis of the performance of five prominent EAs—covariance matrix adaptation evolution strategy (CMA-ES), differential evolution (DE), particle swarm optimization (PSO), evolutionary strategy (ES), and genetic algorithm (GA)—in addressing different continuous control tasks, including 2D robot navigation and modular soft robot control; moreover, as a reference, we include in the comparison a few regression problems, tackled with ANNs as regressors, and a few benchmark problems. Our objective is to evaluate the relative strengths and weaknesses of these EAs in handling the increasing complexities of search spaces inherent in continuous control tasks. To ensure a thorough comparison, we employ two key metrics: search effectiveness, measured by the normalized effectiveness rank (NER) of the best solutions at the last iteration, and search efficiency, which quantifies the number of fitness evaluations required to achieve satisfactory effectiveness. Our experimental results reveal that even the simplest EAs, such as GA and DE, exhibit commendable performance in task achievement. While they may not be comparable to the more sophisticated methods on benchmark problems, their effectiveness in addressing the control tasks is noteworthy. In particular, we find that the two EAs which perform best on control problems, GA and DE, are those which employ a recombination operator: we believe that it is

beneficial since it allows to effectively explore the complex fitness landscapes corresponding to control problems, eventually making these simple EAs effective and efficient for neuroevolution.

2 METHODS

2.1 Evolutionary algorithms

Our study encompasses five well-known EAs for numerical optimization, namely, CMA-ES, DE, PSO, ES, and GA. For the sake of brevity, we refrain from providing detailed descriptions of the inner workings of each EA, except for ES and GA, which boast numerous variants in the existing literature. For all the remaining EAs, we only provide a brief introduction and outline the main parameters values we used in our experiments. If not otherwise specified, we took values from the literature or based on our previous experience.

We remark that our focus is on numerical optimization. Thus, the search space is $\mathbb{R}^p$ and a solution $\mathbf{x} \in \mathbb{R}^p$ is a p -dimensional numerical vector. It is worth noting that all the EAs here considered are designed to work on $\mathbb{R}^p$, with the exception of GA, which is more general and can also work on different search spaces.

CMA-ES [9] is a form of ES which employs, during the optimization process, a mechanism for estimating the distribution of parameters to be optimized. Moreover, CMA-ES adopts a strategy for adaptively changing the step-size, *i.e.*, the magnitude of the step in $\mathbb{R}^p$ between subsequent iterations of the search. CMA-ES has been widely used for solving control problems, also involving the kind of simulated robots we consider in this study [8, 24]. We here adopt the vanilla version of CMA-ES described in [9] where we set all the parameters to their default values.

DE [41] is a simple EA for numerical optimization which iteratively improves candidate solutions by recombining them through crossover. There exist many variants of DE, often identified according to the DE/a/b/c naming scheme. Here we use the DE/rand/1 variant where we set the population size $n_{\text{pop}} = NP = 15$, the differential weight $w_{\text{diff}} = F = 0.5$, and the crossover probability $p_{\text{xo}} = CR = 0.8$. Variants of DE have already been used for evolving ANNs [2], as well as for coping with other design choices in the robotic domain [27].

PSO is a bio-inspired optimization method which, differently from the other EAs here considered, does not root the inspiration in the natural evolution, but rather in the idea of having several particles moving in the search space driven by a local (called cognitive) and a global (called social) force. PSO has been used for both neuroevolution [47] and continuous control [14]. Here we use the standard version of PSO presented in [5], where we set $n_{\text{pop}} = 100$, the cognitive parameter $w = 0.8$, the cognitive acceleration coefficient $\phi_{\text{particle}} = 1.5$, and the social acceleration coefficient $\phi_{\text{global}} = 1.5$.

ES is a rather large family of EAs of which also CMA-ES is a member. To evaluate a simpler member of this family, we also include its basic variant in our analysis, working as follows. We iteratively evolve a population of n_{pop} candidate solutions (*i.e.*, points in $\mathbb{R}^p$) by taking, at each iteration, the n_{parents} best solutions (*i.e.*, those with the best fitness), and computing their mean μ . Then, we produce the next iteration population by taking the current best solution and $n_{\text{pop}} - 1$ new points sampled from $N(\mu, \sigma I)$, where I is the $p \times p$ identity matrix: this way, each new solution is obtained by

applying a zero-mean Gaussian noise with σ variance to each one of its elements. We set $n_{\text{pop}} = 30$ and $n_{\text{parents}} = \lfloor 0.33n_{\text{pop}} \rfloor = 10$. Concerning σ , we experiment with three values (0.02, 0.25, and 0.5), hence resulting in three variants of ES. This straightforward variant of ES has been used, despite its simplicity, for solving control problems with fairly complex modular robots [33], also relying on recurrent and plastic neural controllers [32].

GA is the most general EA of this study, the only one here which can be applied to different search spaces than $\mathbb{R}^p$. We here use a standard variant and customize some of its components (namely, the initialization procedure and the genetic operators) for the case of numerical optimization. Our GA works as follows. At each iteration, we consider the current population of n_{pop} candidate solutions (the parents) and produce an offspring population of n_{pop} solutions by repeating the following steps n_{pop} times: (1) we select crossover (with p_{xo} probability) or mutation (with $1 - p_{\text{xo}}$ probability) as genetic operator; (2) we select one (for mutation) or two (for crossover) parents with a tournament selection of size n_{tour} ; (3) we apply the genetic operator to the selected parent(s) for obtaining a new solution. We then merge the parents and the offspring and select the best n_{pop} solutions, which will hence form the population at the next iteration. We set $n_{\text{pop}} = 100$, $p_{\text{xo}} = 0.8$, and $n_{\text{tour}} = 5$. Concerning the genetic operators, we use the Gaussian mutation, where the offspring $\mathbf{x}'$ comes from the parent $\mathbf{x}$ as $\mathbf{x}' = \mathbf{x} + \epsilon$, with $\epsilon \sim N(0, \sigma I)$, and the segment geometric crossover, where $\mathbf{x}'$ comes from the parents $\mathbf{x}_1, \mathbf{x}_2$ as $\mathbf{x}' = \mathbf{x}_1 + \alpha(\mathbf{x}_2 - \mathbf{x}_1) + \epsilon$, with $\epsilon \sim N(0, \sigma I)$ and $\alpha \sim U([0, 1])$. We experiment with three values of σ (0.02, 0.25, and 0.5), hence resulting in three variants of our GA. Like for the aforementioned ES, also this simple form of GA has been used for evolving neural controllers for modular soft robots [25], also with non-standard forms of ANNs employing attention mechanisms [38].

All the EAs here considered are iterative. In order to have a fair comparison, we set for all the same termination criterion, based on the number of evaluations. We stop an EA run at the first iteration for which it exceeds $n_{\text{eval}} = 10\,000$ fitness evaluations. Moreover, for all the EAs, we randomly initialize the initial population of candidate solutions by sampling the uniform distribution $U([-1, 1])^p$.

We remark that the σ parameter in ES and GA determines the magnitude of the modification applied to a current solution to obtain the next one. That is, σ represents the *step-size*. A large step-size corresponds to a potentially faster exploration of the search space. However, it can lead to a poorer ability to finally focus on a small, promising region of $\mathbb{R}^p$. A small step-size does the opposite. The parameter σ hence plays a key role in determining the exploration vs. exploitation trade-off and taking a value for it is not easy, in general. For these reasons, we decide to experiment with three values for σ for the two most generic EAs which adopt it: ES and GA.

We also remark that GA and DE are the only EAs here employing a proper form of crossover.

2.2 Numerical optimization problems

We consider several numerical optimization problems where the goal is to find the minimum or the maximum (depending on the problem) of a given objective function $f : \mathbb{R}^p \rightarrow \mathbb{R}$.

We group the problems in four scenarios: a *scenario* denotes a group of problems where the candidate solutions, *i.e.*, points in $\mathbb{R}^p$, represent the same kind of entities. In the first scenario, each $\mathbf{x}$ is just a point and the goal is to find the minimum of a function in which $\mathbf{x}$ is the input—that is, we “directly optimize” f . In the other three scenarios, each point is a vector $\boldsymbol{\theta} \in \mathbb{R}^p$ of the numerical parameters defining an ANN: hence, those scenarios actually correspond to neuroevolution problems. For these scenarios, we use $\boldsymbol{\theta}$ instead of $\mathbf{x}$ for a candidate solution, following the common convention for ANN parameters. In the last two scenarios, the ANN is used inside an agent to solve a control problem and the quality of the ANN, hence the fitness $f(\boldsymbol{\theta})$ of the candidate solution $\boldsymbol{\theta}$, is determined through a simulation. The physical environment of the last scenario is more complex than the one of the third.

It can hence be seen that the scenarios differ in how “direct” is the mapping between the solution $\boldsymbol{\theta}$ and its fitness value $f(\boldsymbol{\theta})$: very direct for the first scenario, very indirect for the last scenario. We expect that this difference may have an impact on how the EAs perform in solving the corresponding optimization problems.

2.2.1 Scenario 1: synthetic problems. The first scenario we consider for comparing EAs involves synthetic problems. With synthetic problems we mean a class of optimization problems designed artificially to assess the performance of optimization algorithms [10, 12, 18], with objective functions and search spaces constructed to have some desired problem properties, *e.g.*, a smooth or rugged fitness landscape [19] and one or more local optima [6].

Our aim is to create a collection of problems which is diverse in the main problem properties [18]: unimodal/multimodal, regular/irregular, separable/non-separable, problem size p , and distance of the optimal solution from the origin. We choose the Sphere problem, the point aiming (PA) problem, the circular point aiming (CPA) problem, the Ackley problem, and the Rastrigin problem, each representing different challenges for the EAs.

For all of them, the objective function f is defined parametrically with respect to the size p of the search space and has to be minimized. For all, we considered four variants with p being 20, 100, 200, and 500.

Sphere is a uni-modal problem with an objective function given by $f(\mathbf{x}) = \sum_{i=1}^p x_i^2$. This function has a global optimum at $\mathbf{x}^* = \mathbf{0}$. This is the easiest problem of this study.

PA is a uni-modal problem with $f(\mathbf{x}) = \|\mathbf{x} - \mathbf{x}^*\|$ (where $\|\cdot\|$ is the L^2 or Euclidean norm), with a global optimum in the target point $\mathbf{x}^* \in \mathbb{R}^p$. In other words, PA is simply a generalization of Sphere, with, in general, the optimum not being located in $\mathbf{0}$. We considered two variants of PA with $\mathbf{x}^* = \mathbf{1}$ and $\mathbf{x}^* = \mathbf{10}$ to assess how well the EAs cope with optima which are more or less far from the origin. We recall that in our experiments all the EAs start from a region of the search space which is “around” the origin $\mathbf{0}$.

CPA is a multi-modal variant of PA with $n = 5$ target points which are global optima. For each value of p , we use the same five target points for all the experiments: we obtain them by sampling uniformly in the circumference centered at $\mathbf{1}$ with a radius of 2. The objective function is $f(\mathbf{x}) = \min_{i \in \{1, \dots, 5\}} \|\mathbf{x} - \mathbf{x}_i^*\|$, *i.e.*, the fitness of a point is its distance to the closest target.

Ackley [1] is a multi-modal and non-separable problem with $f(\mathbf{x}) = -20 \exp \left(-0.2 \sqrt{\frac{1}{p} \sum_{i=1}^p x_i^2} \right) - \exp \left(\frac{1}{p} \sum_{i=1}^p \cos(2\pi x_i) \right) + 20 + \exp(1)$.

Finally, Rastrigin [30] is a multi-modal problem characterized by a rugged fitness landscape resulting from the function $f(\mathbf{x}) = 10p + \sum_{i=1}^p (x_i^2 - 10 \cos(2\pi x_i))$.

Both Ackley and Rastrigin have a global optimum at $\mathbf{x}^* = \mathbf{0}$, but they have numerous local optima, potentially misleading search algorithms.

Summarizing, in this scenario we have $24 = 6 \cdot 4$ problems with four different search space sizes, with $p \in \{20, 100, 200, 500\}$. We denote these problems by name- p , with name being the name of the problem.

2.2.2 Scenario 2: ANN-based regression. The second scenario involves regression problems. A regression problem consists in, given a dataset $D = \left\{ \left(\mathbf{x}^{(i)}, y^{(i)} \right) \right\}_i$ with $\mathbf{x}^{(i)} \in \mathbb{R}^m$ and $y^{(i)} \in \mathbb{R}$, finding the function $g : \mathbb{R}^m \rightarrow \mathbb{R}$ which best fits the dataset. Namely, the goal is to find a function g which minimizes the mean squared error (MSE) on D , *i.e.*, $f(g) = \text{MSE}(g, D) = \frac{1}{n} \sum_{i=1}^n \left(g(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$.

Since in this work we focus on neuroevolution, we use ANNs as functions, setting them to have exactly m inputs and one output. We keep the architecture fixed and we optimize the weights $\boldsymbol{\theta}$ of the ANN with the EA under analysis, relying on the MSE as the fitness measure. We employ three architectures, all of them corresponding to feed-forward fully connected ANNs with a single hidden layer, making them different in the size of the hidden layer with respect to the size of the input layer: *i.e.*, m , $2m$, and $3m$. The number of parameters of the ANN is hence $|\boldsymbol{\theta}| = p = (m+1)\rho_h m + (\rho_h m + 1)$, where $\rho_h \in \{1, 2, 3\}$ is the relative size of the hidden layer and the +1 represents the bias. For all the neurons, we utilize the tanh activation function.

We consider three real-world datasets, chosen according to the considerations of White et al. [45]. Specifically, we include the Concrete dataset ($m = 8$, $|D| = 825$), the Energy efficiency dataset ($m = 8$, $|D| = 615$, from now on simply Energy), and the Wine dataset ($m = 11$, $|D| = 3919$). For all the datasets, we standardize the independent variables, *i.e.*, the $\mathbf{x}^{(i)}$, and we rescale the dependent value in $[-1, 1]$, in order to make the $y^{(i)}$ actually fittable by an ANN with tanh as activation function.

Summarizing, in this scenario we have $9 = 3 \cdot 3$ problems with six different search space sizes, ranging from $p = 81$ (for Concrete and Energy with $\rho_h = 1$) to $p = 430$ (for Wine with $\rho_h = 3$). We denote these problems by name- ρ_h , with name being the name of the dataset and ρ_h being the relative size of the hidden layer.

2.2.3 Scenario 3: ANN-based 2D navigation. The third scenario involves 2D navigation tasks, performed in simulation. The agent is a differential-drive wheeled robot with a circular shape and two wheels on its sides. Its goal is to reach a target position in an arena with obstacles.

The robot has two control outputs regulating the velocity of the two wheels. Regarding the input, *i.e.*, the agent perception, the robot is equipped with m proximity sensors evenly distributed around half of its circumference (the frontal part), along with two

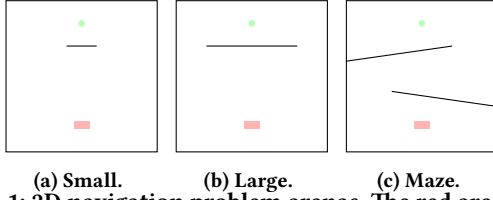


Figure 1: 2D navigation problem arenas. The red area represents the random starting point of the robot, the green circle the position of the target, the black lines the boundaries and the obstacles of the arena.

sensors perceiving the distance and the angle to the target position. In detail, we use an arena of $1\text{ m} \times 1\text{ m}$ size and a robot with a radius of 5 cm, proximity sensors with a range of 1 m (i.e., a sensors reads 1 when no obstacles are within the range and reads 0 when it is in direct contact with an obstacle), and maximum speed of 10 cm s^{-1} . At the beginning of the simulation, we place the robot at (x_0, y_0) , with $x_0 \sim U([0.45, 0.55])$ and $y_0 \sim U([0.80, 0.85])$ —the fitness f is hence stochastic. The target is at $(0.50, 0.15)$. We perform a discrete time simulation with time step $\delta t = 0.1\text{ s}$ and duration of 60 s. We consider three variants of the 2D navigation problem, based on three different arenas: one with a small barrier in front of the target, one with a larger barrier, and one with two barriers forming a simple maze. We show the three arenas in Figure 1.

We consider three robot setups differing in the number m of proximity sensors, with $m \in \{3, 5, 9\}$. Thus, we wish to find a control function $g : \mathbb{R}^{m+2} \rightarrow \mathbb{R}^2$, i.e., a *policy*, that drives the agent to the goal. As before, we implement g with an ANN and optimize its weights $\theta \in \mathbb{R}^P$ with an EA. However, in this scenario we employ a fixed ANN architecture with an input layer with $m + 2$ neurons, an output layer with 2 neurons, and a single hidden layer with $3(m + 2)$ neurons, all with tanh as activation function.

In all the cases, the fitness function f is the average distance between the agent and the target position during the entire simulation, to be minimized.

Summarizing, in this scenario we have $9 = 3 \cdot 3$ problems with three different search space sizes, with $p \in \{122, 212, 464\}$. We denote these problems by name- m , with name being the name of the arena and m being the number of proximity sensors.

2.2.4 Scenario 4: ANN-based voxel-based soft robot (VSR) control. The last scenario concerns the control of VSRs [16, 20], also known as virtual soft robots [28], a family of modular soft robots composed of elastic cubes (*voxels*) which can contract and expand to achieve movement, similarly to muscular tissues. We consider a 2D variant in which voxels are actually squares, for ease of simulation. In particular, we experiment with the model of 2D VSRs proposed in [23], where each voxel is modeled as four masses at the corners connected by several spring-damper systems to simulate softness.

A VSR is defined by its *morphology* (body), and *controller* (brain). The morphology is polyomino, i.e., a grid arrangement of voxels that also determines type and allocation of the sensors—the morphology of the VSRs may be optimized as well as the controller [31], but we here focus only on the latter. We consider two morphologies, a Biped and a Tower, composed respectively of 10 and 14 voxels—we show them in Figure 2. Concerning the sensors, we employ the following four types of sensors: (a) proximity, which read the

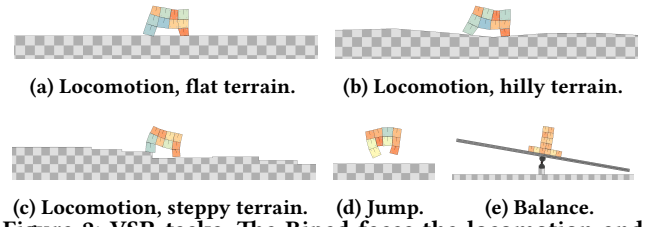


Figure 2: VSR tasks. The Biped faces the locomotion and jump tasks; the Tower faces the balance task. The color of each voxel represents its current area ratio: orange for < 1 , blue for > 1 .

distance to the closest object (up to a range r) in a given direction α ; (b) area, which read the current voxel area ratio with respect to the rest area, (c) velocity, which read the voxel velocity along either the x or y axis integral with the voxel, (d) sin, a “fake” sensor which outputs a sinusoidal signal with 1 Hz frequency. We consider two distinct sensor configurations: a homogeneous configuration, with sensors of type area and velocity (along both axes) in each voxel, and a heterogeneous configuration, with sensors of type sin and proximity (with $r = 5\text{ m}$ and $\alpha = -\frac{\pi}{12}$, the side of the voxel is 1 m long) in the “head” of the VSR, i.e., in its topmost-rightmost voxel, sensors of type area and velocity (both axes), in the topmost row of voxels, and proximity (with $r = 1\text{ m}$ and $\alpha = -\frac{\pi}{2}$) sensors in the bottom row of voxels. We normalize the output of each sensor making it fall in $[-1, 1]$.

The controller of a VSR manages voxel contraction and expansion, processing sensor readings through an ANNs. We consider three controllers, a centralized (C) controller [32, 42], consisting of a single ANN taking as input all the sensor readings and producing as output one contraction value in $[-1, 1]$ for each voxel, and two distributed controllers [22], where each voxel has its own ANN taking local sensor readings, giving as output the local contraction value, and exchanging data with its neighbor voxels (we refer the reader to [22] for further details). The two distributed variants differ in the homogeneity of the ANNs employed in the voxels. In the homo-distributed (HoD) controller we use the same network (architecture and parameters) for all the voxels; in the and hetero-distributed (HeD) controller, we use different networks. Clearly, HoD is not compatible with the heterogeneous sensor configuration, since the number of sensors in each voxel is not the same; on the other hand, using many ANNs corresponds to searching over a larger search space where each point is the concatenation of all the ANNs parameters. We rely on C and HeD for the VSRs with the heterogeneous sensor configuration, whereas we employ HoD in conjunction with the homogeneous sensor configuration. In every case, we use ANNs with one single hidden layer with the same size as the input layer, the tanh activation function in every neuron, and apply a small Gaussian noise with 0.05 variance on each sensor reading. Moreover, we actually invoke the ANN-based controller every 0.2 s, instead of at each time step, to avoid vibrational behaviors. Between subsequent invocations, we keep the output constant. To optimize the ANNs that control the VSRs, we flatten their weights and evolve the resulting vector with the EAs. For HeD we concatenate the vectors of all the ANNs of the VSR. We hence optimize controllers having 52, 280, and 374 parameters

for the Biped and 52, 275, and 420 for the Tower, respectively for HoD, C, and HeD.

We consider the VSR control tasks of locomotion, jump, and balance. For locomotion, we consider three variants differing in the terrain: a flat one, a hilly one, and a steppy one, *i.e.*, a piece-wise flat terrain with a varying altitude (see Figure 2). In the locomotion tasks, the VSR has to move as far as possible from its initial position along the x -axis positive direction: the fitness is the average velocity v_x of the center of mass of the VSR in a simulation lasting 30 s, to be maximized. In the jump task, the VSR has to maximize its relative distance from the ground: the fitness is the maximum height of the center of mass of the VSR to the ground in a simulation lasting 10 s, discarding the first 5 s, to be maximized. In the balance task, we place the VSR on a 30 m-long swing, with a joint at its center, and the robot objective is to keep the swing balanced: the fitness, to be minimized, is the average angle of the swing in a simulation lasting 30 s, with a malus for each time step in which the swing touches the ground. We perform all the simulations with the simulator presented in [23] leaving all parameters to default values and with a time step of $\frac{1}{60}$ s. Figure 2 provides a visual representation of the five tasks. We used the Tower for the balancing task and the Biped for all the other four tasks.

Summarizing, in this scenario we have $15 = 5 \cdot 3$ problems (resulting from the combination of the task and the controller) with five different search space sizes, ranging from $p = 52$ to $p = 420$. We denote these problems by task-controller, with name being the short name of the task, controller being the type of controller.

2.3 Comparison indexes

2.3.1 Search effectiveness. To compare the EAs in terms of search effectiveness, we use the fitness $f(\mathbf{x}^*)$ of the best solution $\mathbf{x}^*$ of the population at the last iteration, *i.e.*, the one with the maximum or minimum fitness, depending on the problem.

Since fitness values are in general not comparable across problems and scenarios, we also consider the NER. Given one problem and a set of n_{EAs} EAs: (1) we apply each EA to the problem n_{rep} times, collecting n_{rep} samples for the final best fitness $f(\mathbf{x}^*)$; (2) we sort all the collected final best fitness values and assign to each pair (EA, $f(\mathbf{x}^*)$) a rank, 1 for the best, $n_{EAs}n_{rep}$ for the worst; (3) we normalize all the ranks; finally, (4) we associate with each run of an EA as NER its normalized rank in the problem. For all cases, $NER \in [0, 1]$, with 0 meaning that the run of an EA scored comparatively well in the problem.

For giving an even higher level assessment of the performance of an EA on several problems, we also consider the number of victories score (NoVS). Given one problem, a set of n_{EAs} EAs, and their corresponding $n_{EAs}n_{rep}$ final best fitness measures, we (1) determine the best EA on the problem as the one with the best median $f(\mathbf{x}^*)$ and (2) say that an EA scores a victory on the problem if its $f(\mathbf{x}^*)$ is not statistically significantly different from the one of the best EA. We check statistical significance using the Wilcoxon signed ranks test [7], with the null hypothesis that the two samples have the same mean. To mitigate the family-wise error rate due to the multiple pairwise tests performed, we set a significance level of $\alpha = 0.01$. Finally, given the problems of a scenario, we define the NoVS of an EA as the number of problems for which it scores a victory.

2.3.2 Search efficiency. We here consider an EA to be efficient based on how many fitness evaluations it requires to reach a satisfactory effectiveness. Given a problem, a set of n_{EAs} EAs, and their corresponding $n_{EAs}n_{rep}$ final best fitness measures, we (1) determine the third quartile f_{75} of all the fitness measures (sorting them according to the problem goal) and (2) consider for each run of each EA on the problem the number of fitness evaluations it takes to obtain a solution with a fitness not worse than f_{75} . We call this number the evals to 3-rd quartile (EtTQ), for which the lower, the better; we remark that we set the same computational budget of $n_{evals} = 10\,000$ fitness evaluations for all the problems and EAs. If a run of an EA never obtains a good enough solution, *i.e.*, if it never achieves the f_{75} for a problem, we set its EtTQ to n_{evals} .

3 EXPERIMENTS AND RESULTS

We compared nine EAs, some of them being the same EA with different step-sizes: CMA-ES, DE, PSO, ES-0.02, ES-0.25, ES-0.5, GA-0.02, GA-0.25, and GA-0.5. We applied them to $24+9+9+15 = 57$ numerical optimization problems, grouped in the four scenarios described above. For each (EA, problem) pair, we performed $n_{rep} = 30$ (20 for the problems of scenario 4, which are computationally heavier) executions (*i.e.*, runs) of the EA, by varying the random generator seed. We totaled a number of $9 \cdot 42 \cdot 30 + 9 \cdot 15 \cdot 20 = 14\,040$ runs.

We performed the experiments using the evolutionary computation (EC) software framework JGEA [26] on a pair of computers running the Ubuntu 20.04 LTS operating system and equipped with an Intel(R) Xeon(R) W-2295 CPU clocked at 3.0 GHz, 36 cores, and 64 GB of RAM. We used the computers non-exclusively: for this reason, we do not focus on the execution times in the discussion of the results. However, for completeness we report that a run on a synthetic problem (scenario 1) took 1 s to 5 s; a run on an ANN-based regression problem (scenario 2) took approximately 75 s for the Concrete and Energy datasets and 250 s for the Wine dataset; a run on an ANN-based 2D navigation problem (scenario 3) took 100 s to 200 s; and a run on an ANN-based VSR control (scenario 4) took approximately 10 000 s for the balance and locomotion tasks and 2 000 s for the jump task.

We made both the code of our experiments and the intermediate results publicly available at https://github.com/elsalibymichel/2024_NEWK-GECCO_EA-Comparison.

3.1 Scenario 1: synthetic problems

In Figure 3, we show the distributions of the final best fitness $f(\mathbf{x}^*)$ across 30 runs for each synthetic problem individually, all of which are minimization problems. CMA-ES emerges as the most effective and efficient EA, as corroborated by the distributions of NER and EtTQ in Figure 4. This result holds also when considering the NoVS within the synthetic problem scenario, as we show in Table 1, where CMA-ES reaches the higher score, scoring a victory in 18 of the 24 synthetic problems.

The relation observed between p and NER, averaged for different problems with the same p , shown in Figure 4, suggests that the dimensionality of the search space may have limited significance in determining the ranking of EAs—with the exception of PSO and, more slightly, CMA-ES. Instead, we hypothesize that the

ruggedness of the fitness landscape plays a more pivotal role in determining effectiveness. In problems involving more irregular fitness landscapes, such as the Ackley and Rastrigin functions, the performance of CMA-ES diminishes, particularly as the dimension of the search space increases. In such cases, GA-0.02 effectiveness is better than that of CMA-ES: in Table 1, we present the NoVS attained by GA-0.02 within the synthetic problem context, which amounts to 6 and corresponds to the four Rastrigin problems as well as Ackley-500 and Sphere-500.

Looking at the individual problems and focusing on GA, it can be seen that for those problems having an optimum in 0, the smaller the step-size σ , the better. Conversely, for PA-1- p and PA-10- p , which have the optimum far from the origin, the greater the step-size, the better, regardless of the value of p . This consideration holds, but less evidently, also for ES.

3.2 Scenario 2: ANN-based regression

Figures 5 and 6 show the results for the ANN-based regression problems, in the same order and with the same visual syntax of Figures 3 and 4 in the previous section. On regression problems, CMA-ES is among the worst performing EAs, evidenced also by its NoVS of 0 (see Table 1). In contrast, DE achieves the greatest NoVS, scoring a victory in 7 out of the 9 problems. This observation holds even when examining the individual problems.

When we analyze the relation between p and the NER, shown Figure 6, it becomes apparent that as the dimensionality of the genotype space increases, GA-0.02, PSO, and ES-0.02 exhibit higher (*i.e.*, worse) NER values compared to DE. Specifically, GA-0.02 achieves a NoVS of 4 (see Table 1) corresponding to ANN architectures with a number of hidden neurons three times that of the inputs for all the three datasets, *i.e.*, Concrete-3, Energy-3, and Wine-3, and twice that of the input for the Wine dataset, *i.e.*, Wine-2. PSO gets a NoVS of 1, on Concrete-3. ES-0.02 achieves a NoVS of 3, all on the Concrete- ρ_h problems.

Finally, regarding efficiency, Figure 6 shows that GA-0.02 and ES-0.02 are the best EAs. More broadly, GA-0.02, GA-0.25, ES-0.02, ES-0.25, DE, and PSO appear largely more consistent (*i.e.*, with narrower distributions of EtTQ) than the remaining EAs.

3.3 Scenario 3: ANN-based 2D navigation

Figures 7 and 8 show the results for the ANN-based 2D navigation problems. On these problems, the differences between EAs are less apparent than in the previous two scenarios. Moreover, the harder the problem (Small- m being the easiest, Maze- m being the hardest), the more negligible the differences, with the exception of ES-0.02, which performs badly and inconsistently in all cases. All to all, Figure 8 shows that, in terms of search effectiveness, DE is again the best EA, followed by the variants of GA and ES with medium and large step-size. These findings are consistent with the NoVSs reported in Table 1, where we find that DE wins in 9 problems, followed by GA-0.5 with a NoVS of 5, and GA-0.25 with 4.

The general observation concerning efficiency and impact of p on NER resemble those made on effectiveness: the differences between EAs are less sharp than in the previous scenarios.

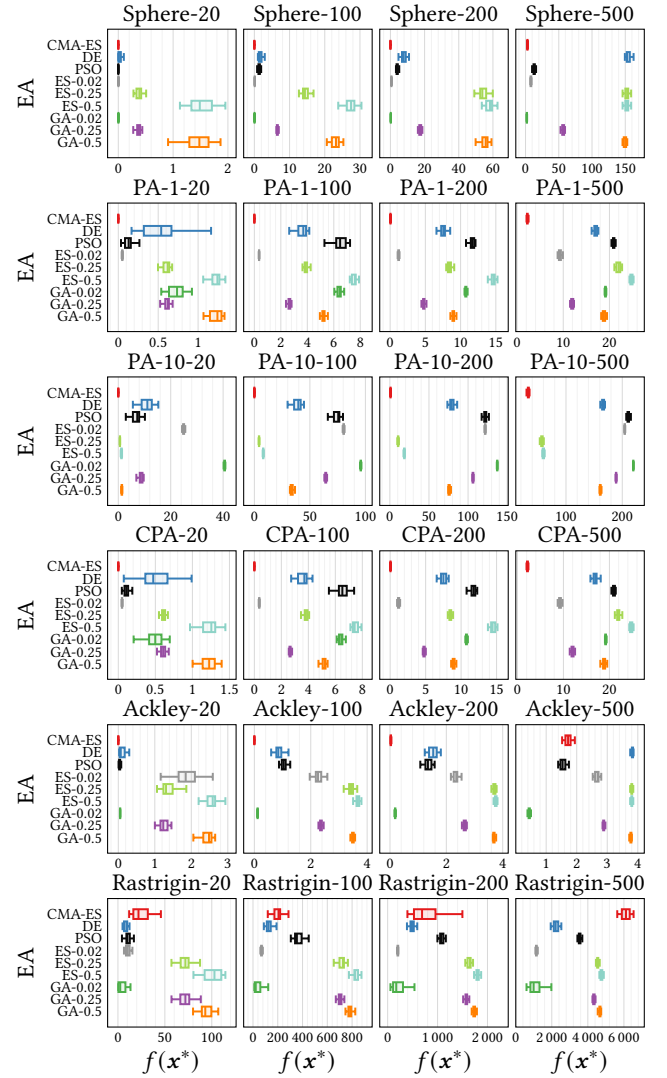


Figure 3: Final best fitness $f(x^*)$ distributions across the 30 runs for the synthetic problems (scenario 1). For all the problems, the lower $f(x^*)$, the better.

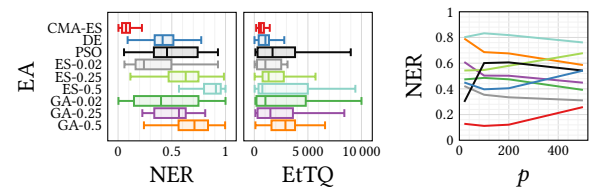


Figure 4: NER and EtTQ distributions across all the runs and NER vs. p for the synthetic problems (scenario 1).

3.4 Scenario 4: ANN-based VSR control

Figures 9 and 10 show the results for the ANN-based VSR control problems. We remark that in this scenario we have both minimization and maximization problems. The former includes the three balance problems, while the latter comprises all the other problems.

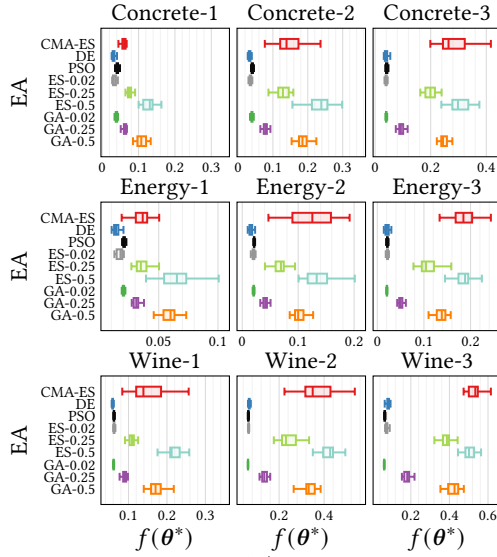


Figure 5: Final best fitness $f(\theta^*)$ distributions across the 30 runs for the regression problems (scenario 2). For all the problems, the lower $f(\theta^*)$, the better.

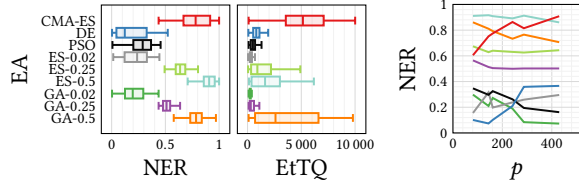


Figure 6: NER and EtTQ distributions across all the runs and NER vs. p for the regression problems (scenario 2).

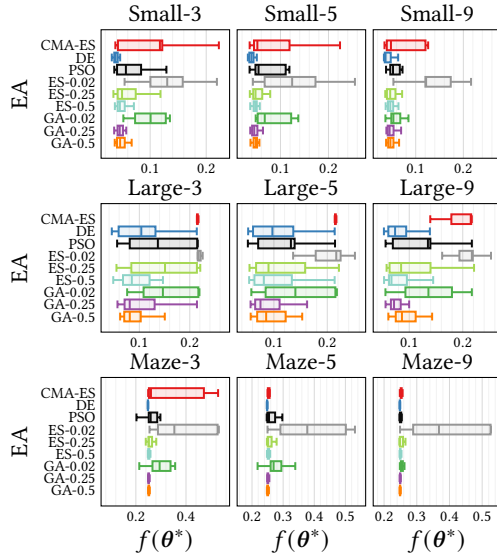


Figure 7: Final best fitness $f(\theta^*)$ distributions across the 30 runs for the 2D navigation problems (scenario 3). For all the problems, the lower $f(\theta^*)$, the better.

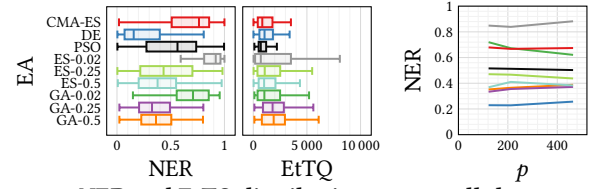


Figure 8: NER and EtTQ distributions across all the runs and NER vs. p for the 2D navigation problems (scenario 3).

When we analyze the final best fitness and the NER distributions and the NER, we find that DE and GA-0.5 are the EAs that have the best overall results. However, when we consider the efficiency, DE and CMA-ES demonstrate notably superior results compared to GA-0.5.

If we focus on the relation between p and the NER shown in Figure 10, we observe that as in the 2D navigation scenario, there is not a strong impact of p . However, in the VSR scenario, the NER is far from being constant for all the values of p . The sharp discontinuity is from $p = 275$ (Tower with C controller) to $p = 280$ (Biped with C controller): besides the difference in morphology, which we deem not particularly relevant based on our experience, the two cases differ in the task being faced, since the Tower VSR is used in the Balance task and the Biped faces the other three tasks.

When analyzing NoVS, shown in Table 1, we find results that are consistent with what we have observed so far for scenarios 2 and 3. DE achieves the highest NoVS, winning in 13 out of 15 VSR problems, falling short only in Balance-C and Flat-C. GA-0.5 and GA-0.25 attain a NoVS of 12 and 11, respectively. Although GA-0.5 falters in Balance-C, Balance-HoD, and Flat-HoD, and GA-0.25 struggles with Balance-HeD, Jump-HeD, Jump-HoD, and Steppy-HeD, for all the problems, at least one of these algorithms achieves the best or non-worse-than-best effectiveness.

Finally, we observe that CMA-ES achieves better results in this scenario than in the regression and 2D navigation scenarios. CMA-ES achieves the best median effectiveness in one problem, Jump-HeD, see Figure 9. It also achieves a NoVS of 5, see Table 1, winning in Flat-HeD, Flat-C, Hilly-C, Jump-HeD, and Jump-C. Notably, CMA-ES appears to struggle in the problems involving the HoD controller: we recall that in these cases, there is the same ANN in every voxel of the VSR. We hypothesize that this makes the dynamics of the overall system particularly complex and, hence, the fitness landscape particularly rugged.

3.5 Discussion

We show in Table 1 the NoVS for each EA across the four scenarios. This table summarizes the main results of our study for what concerns search effectiveness.

The main finding is that, on the three scenarios where the goal is to evolve the weights of an ANN, *i.e.*, those corresponding to neuroevolution, DE consistently scores the largest NoVS, followed by some variants of simple EAs as GA or ES.

We also observe that the performance of an EA on synthetic problems is not predictive of the performance of that EA on more complex neuroevolution problems. While we do acknowledge that we have not attempted to build a large set of synthetic benchmark problems with the specific aim of telling apart EAs for numerical

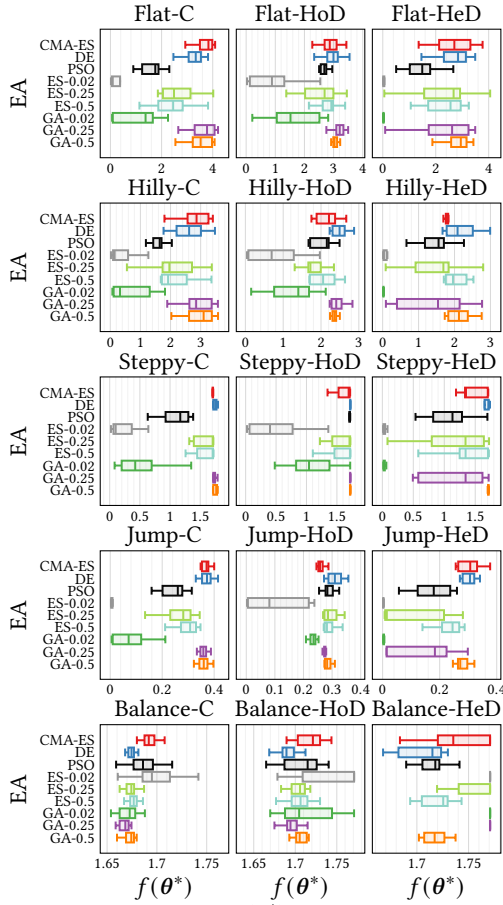


Figure 9: Final best fitness $f(\theta^*)$ distributions across the 30 runs for the VSR control problems (scenario 4). For the three balance problems, the lower $f(\theta^*)$, the better. For all other problems, the higher $f(\theta^*)$, the better.

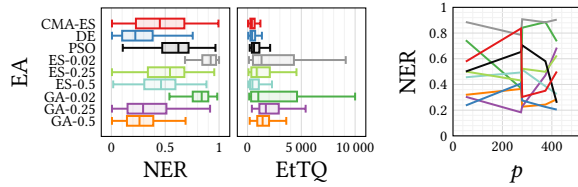


Figure 10: NER and EtTQ distributions across all the runs and NER vs. p for the VSR control problems (scenario 4).

Table 1: The NoVS of each EA across the scenarios.

EA	Synthetic	Regression	2D nav.	VSR
CMA-ES	18	0	0	5
DE	0	7	9	13
PSO	0	1	2	2
ES-0.02	2	3	0	0
ES-0.25	0	0	3	1
ES-0.5	0	0	3	3
GA-0.02	6	4	0	1
GA-0.25	0	0	4	11
GA-0.5	0	0	5	12

optimization, we remark that we chose a decently diverse set of neuroevolution problems.

Another less apparent finding concerns the complexity of the neuroevolution problems. Characterizing quantitatively such a property is beyond the scope of this paper. Nevertheless, we remark that the way the fitness $f(\theta)$ is computed out of a candidate solution θ is more direct for the problems of scenario 2 and least direct for those of scenario 4, due to the complex dynamics induced by VSRs softness and, in some cases, by the distributed nature of their controller. Interestingly, GA performs better in more complex cases, with GA-0.5 achieving a NoVS of 12, just one victory lower than DE.

We hypothesize, also supported by the observations done on individual problems of scenario 1, that the complexity of a control problem, *i.e.*, the degree to which the mapping between a solution and its fitness is direct, impacts the ruggedness of the fitness landscape. On very rugged landscapes, GA, in particular with a large step-size, exploits its ability to “quickly jump” from one to another region of the search space for finding good solutions.

Finally, we note that the two best EAs on the ANN-based control problems are DE and GA, the only EAs of this comparison which employ a crossover operator, *i.e.*, which build new solutions by recombining existing solutions which may be very far away from each other. We believe that this mechanism is particularly beneficial on the rugged fitness landscapes resulting from complex control problems, acting approximately like a very large step in the search space. Indeed, the fact that the crossover operator helps the search when facing rugged landscapes is not new [19, 34].

4 CONCLUDING REMARKS

In this study, we compared the behavior of various EAs when used in neuroevolution for continuous control tasks. Our aim was to gain more insights into the characteristics of the problem and the EA that affect performance, either positively or negatively. We compared 9 EAs applied in 24 + 9 + 9 + 15 problems, which we divided into four scenarios. Three of the scenarios were forms of neuroevolution, two of them being control tasks. The scenarios had increasing complexity and the difference between the two control scenarios was due to the intrinsically rich dynamics of the considered (simulated) robotic agents.

Our experimental results show that simpler EAs like DE and GA achieve comparatively good effectiveness and efficiency performance, particularly when dealing with more complex problems. We hypothesize that the effectiveness of these simpler EAs is due to their enhanced exploration capabilities and, in particular, due to their use of crossover.

Furthermore, we hypothesize that the ruggedness of the fitness landscape plays a crucial role in the comparative performance of the EAs. Thus, we intend to further investigate how the fitness landscape is affected by the problem characteristics, *e.g.*, observation- and action-space dimension, randomness of the environment, presence/absence of a state in the controller, deceptiveness of the goal, and so on [44]. Another question that arises from our results is whether the contrast between individual learning, as for the case of imitation learning [25], and social learning [3, 15] may be similar to that between an EA without or with crossover.

ACKNOWLEDGMENTS

This study was carried out within the PNRR research activities of the consortium iNEST (Interconnected North-Est Innovation Ecosystem) funded by the European Union Next-GenerationEU (Piano Nazionale di Ripresa e Resilienza (PNRR) - Missione 4 Componente 2, Investimento 1.5 – D.D. 1058 23/06/2022, ECS_00000043).

REFERENCES

- [1] David Ackley. 2012. *A connectionist machine for genetic hillclimbing*. Vol. 28. Springer science & business media.
- [2] Marco Baioletti, Gabriele Di Bari, Alfredo Milani, and Valentina Poggioni. 2020. Differential evolution for neural networks optimization. *Mathematics* 8, 1 (2020), 69.
- [3] Alberto Bartoli, Marco Catto, Andrea De Lorenzo, Eric Medvet, and Jacopo Talamini. 2020. Mechanisms of social learning in evolved artificial life. In *Artificial Life Conference Proceedings* 32. MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info ..., 190–198.
- [4] Lucian Buşoniu, Tim De Bruin, Domagoj Tolić, Jens Kober, and Ivana Palunko. 2018. Reinforcement learning for control: Performance, stability, and deep approximators. *Annual Reviews in Control* 46 (2018), 8–28.
- [5] Maurice Clerc. 2012. Beyond standard particle swarm optimisation. In *Innovations and Developments of Swarm Intelligence Applications*. IGI Global, 1–19.
- [6] Swagatam Das, Sayan Maity, Bo-Yang Qu, and Ponnuthurai Nagarathnam Suganthan. 2011. Real-parameter evolutionary multimodal optimization—A survey of the state-of-the-art. *Swarm and Evolutionary Computation* 1, 2 (2011), 71–88.
- [7] Joaquín Derrac, Salvador García, Daniel Molina, and Francisco Herrera. 2011. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation* 1, 1 (2011), 3–18.
- [8] Andrea Ferigo, Eric Medvet, and Giovanni Iacca. 2022. Optimizing the sensory apparatus of voxel-based soft robots through evolution and babbling. *SN Computer Science* 3, 2 (2022), 109.
- [9] Nikolaus Hansen. 2016. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772* (2016).
- [10] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. 2021. COCO: A platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software* 36, 1 (2021), 114–144.
- [11] Matthew Hausknecht, Joel Lehman, Risto Miikkiläinen, and Peter Stone. 2014. A neuroevolution approach to general atari game playing. *IEEE Transactions on Computational Intelligence and AI in Games* 6, 4 (2014), 355–366.
- [12] Yifan He and Claus Aranha. 2024. Evolving Benchmark Functions to Compare Evolutionary Algorithms via Genetic Programming. *arXiv preprint arXiv:2403.14146* (2024).
- [13] Verena Heidrich-Meisner and Christian Igel. 2009. Neuroevolution strategies for episodic reinforcement learning. *Journal of Algorithms* 64, 4 (2009), 152–168.
- [14] Daniel Hein, Alexander Hentschel, Thomas A Runkler, and Steffen Udluft. 2016. Reinforcement learning with particle swarm optimization policy (PSO-P) in continuous state and action spaces. *International Journal of Swarm Intelligence Research (IJSIR)* 7, 3 (2016), 23–42.
- [15] Masahiko Higashi, Reiji Suzuki, and Takaya Arita. 2018. The role of social learning in the evolution on a rugged fitness landscape. *Frontiers in Physics* 6 (2018), 88.
- [16] Jonathan Hiller and Hod Lipson. 2011. Automatic design and manufacture of soft robots. *IEEE Transactions on Robotics* 28, 2 (2011), 457–466.
- [17] Seyed Mohammad Jafar Jalali, Sajad Ahmadian, Abbas Khosravi, Seyedali Mirjalili, Mohammad Reza Mahmoudi, and Saeid Nahavandi. 2020. Neuroevolution-based autonomous robot navigation: A comparative study. *Cognitive Systems Research* 62 (2020), 35–43.
- [18] Momin Jamil and Xin-She Yang. 2013. A literature survey of benchmark functions for global optimisation problems. *International Journal of Mathematical Modelling and Numerical Optimisation* 4, 2 (2013), 150–194.
- [19] Krasimir Kolarov. 1997. Landscape ruggedness in evolutionary algorithms. In *Proceedings of 1997 IEEE International Conference on Evolutionary Computation (ICEC'97)*. IEEE, 19–24.
- [20] Julie Legrand, Seppe Terryn, Ellen Roels, and Bram Vanderborght. 2023. Reconfigurable, multi-material, voxel-based soft robots. *IEEE Robotics and Automation Letters* 8, 3 (2023), 1255–1262.
- [21] Horia Mania, Aurelia Guy, and Benjamin Recht. 2018. Simple random search of static linear policies is competitive for reinforcement learning. In *Advances in Neural Information Processing Systems*. S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2018/file/7634ea65a4e6d9041cfd3f7de18e334a-Paper.pdf
- [22] Eric Medvet, Alberto Bartoli, Andrea De Lorenzo, and Giulio Fidel. 2020. Evolution of distributed neural controllers for voxel-based soft robots. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*. 112–120.
- [23] Eric Medvet, Alberto Bartoli, Andrea De Lorenzo, and Stefano Seriani. 2020. 2D-VSR-Sim: A simulation tool for the optimization of 2-D voxel-based soft robots. *SoftwareX* 12 (2020), 100573.
- [24] Eric Medvet, Alberto Bartoli, Federico Pigozzi, and Marco Rochelli. 2021. Biodiversity in evolved voxel-based soft robots. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 129–137.
- [25] Eric Medvet and Giorgia Nadizar. 2024. GP for Continuous Control: Teacher or Learner? The Case of Simulated Modular Soft Robots. In *Genetic Programming Theory and Practice XX*. Springer, 203–224.
- [26] Eric Medvet, Giorgia Nadizar, and Luca Manzoni. 2022. JGEA: a Modular Java Framework for Experimenting with Evolutionary Computation. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. 2009–2018.
- [27] Eric Medvet, Stefano Seriani, Alberto Bartoli, and Paolo Gallina. 2020. Evolutionary optimization of sliding contact positions in powered floor systems for mobile robots. *at-Automatisierungstechnik* 68, 2 (2020), 97–109.
- [28] Alican Mertan and Nick Cheney. 2024. Investigating Premature Convergence in Co-optimization of Morphology and Control in Evolved Virtual Soft Robots. In *European Conference on Genetic Programming (Part of EvoStar)*. Springer.
- [29] David E. Moriarty, Alan C. Schultz, and John J. Grefenstette. 1999. Evolutionary algorithms for reinforcement learning. *Journal of Artificial Intelligence Research* 11 (1999), 241–276.
- [30] Heinz Mühlenbein, M. Schomisch, and Joachim Born. 1991. The parallel genetic algorithm as function optimizer. *Parallel computing* 17, 6-7 (1991), 619–632.
- [31] Jessica Mégane, Eric Medvet, Nuno Lourenço, and Penousal Machado. 2024. Grammar-based Evolution of Polyominoes. In *European Conference on Genetic Programming (Part of EvoStar)*. Springer.
- [32] Giorgia Nadizar, Eric Medvet, Stefano Nichele, and Sidney Pontes-Filho. 2023. An experimental comparison of evolved neural network models for controlling simulated modular soft robots. *Applied Soft Computing* (2023), 110610.
- [33] Giorgia Nadizar, Eric Medvet, Kathryn Walker, and Sebastian Risi. 2023. A Fully-Distributed Shape-Aware Neural Controller for Modular Robots. In *Proceedings of the Genetic and Evolutionary Computation Conference (Lisbon, Portugal) (GECCO '23)*. Association for Computing Machinery, New York, NY, USA, 184–192.
- [34] Simon Naumov. 2023. Empirical Analysis of Crossover-Based Evolutionary Algorithms on Rugged Landscape. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*. 2370–2373.
- [35] Paolo Pagliuca, Nicola Milano, and Stefano Nolfi. 2020. Efficacy of modern neuro-evolutionary strategies for continuous control optimization. *Frontiers in Robotics and AI* 7 (2020), 98.
- [36] Felice Andrea Pellegrino, Franco Blanchini, Gianfranco Fenu, and Erica Salvato. 2022. Closed-loop Control from Data-Driven Open-Loop Optimal Control Trajectories. In *2022 European Control Conference (ECC)*. 1379–1384. <https://doi.org/10.23919/ECC55457.2022.9838344>
- [37] Felice A. Pellegrino, Franco Blanchini, Gianfranco Fenu, and Erica Salvato. 2023. Data-driven dynamic relatively optimal control. *European Journal of Control* 74 (2023), 100839. <https://doi.org/10.1016/j.ejcon.2023.100839> 2023 European Control Conference Special Issue.
- [38] Federico Pigozzi, Yujin Tang, Eric Medvet, and David Ha. 2022. Evolving modular soft robots without explicit inter-module communication using local self-attention. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 148–157.
- [39] Christiaan J Pretorius, Mathys C Du Plessis, and John W Gonsalves. 2017. Neuroevolution of inverted pendulum control: a comparative study of simulation techniques. *Journal of Intelligent & Robotic Systems* 86 (2017), 419–445.
- [40] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. 2017. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864* (2017).
- [41] Rainer Storn and Kenneth Price. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization* 11 (1997), 341–359.
- [42] Jacopo Talamini, Eric Medvet, Alberto Bartoli, and Andrea De Lorenzo. 2019. Evolutionary synthesis of sensing controllers for voxel-based soft robots. In *Artificial life conference proceedings*. MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info ..., 574–581.
- [43] Matthew E Taylor, Shimon Whiteson, and Peter Stone. 2006. Comparing evolutionary and temporal difference methods in a reinforcement learning domain. In *Proceedings of the 8th annual conference on Genetic and evolutionary computation*. 1321–1328.
- [44] Sarah L Thomson, Léni K Le Goff, Emma Hart, and Edgar Buchanan. 2024. Understanding fitness landscapes in morpho-evolution via local optima networks. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '24)*. Association for Computing Machinery, New York, NY, USA.
- [45] David R White, James McDermott, Mauro Castelli, Luca Manzoni, Brian W Goldman, Gabriel Kronberger, Wojciech Jaśkowski, Una-May O'Reilly, and Sean Luke. 2013. Better GP benchmarks: community survey results and proposals. *Genetic Programming and Evolvable Machines* 14 (2013), 3–29.

- [46] Darrell Whitley, Stephen Dominic, Rajarshi Das, and Charles W Anderson. 1993. Genetic reinforcement learning for neurocontrol problems. *Machine Learning* 13 (1993), 259–284.
- [47] Bo Zhao, Fangchao Luo, Haowei Lin, and Derong Liu. 2021. Particle swarm optimized neural networks based local tracking control scheme of unknown nonlinear interconnected systems. *Neural Networks* 134 (2021), 54–63.